GEbPDCM + LTPM

Study / Prep Guide -- Every File, Every Design Decision, From Scratch

Covers both codebases: v2 (src/.py, pure Python) and v1 (ns3/gebpdcm.cc, real NS-3). Read this alongside the source files -- section headers match filenames and function names exactly so you can jump straight to the code being discussed.*

0. How to Use This Guide

This document explains every meaningful line of code in the project, and -- more importantly -- why each design choice was made the way it was, not some other way. It is written for viva/exam prep: if you can answer the "why this, not that" question for every bolded decision below, you can defend the project under questioning.

Structure: Section 1 restates the base paper's problem and equations in plain English (no code yet). Sections 2-8 walk through src/*.py file by file (v2's engine, and v1's design reference). Section 9 walks through ns3/gebpdcm.cc, the real NS-3 port (v1's engine). Section 10 is a running list of every fork in the road this project actually faced, with the option taken and the option rejected. Section 11 is anticipated viva questions with model answers.

1. The Paper, in Plain English

Base paper: Sandhya S. V., S. M. Joshi, "An optimized congestion control protocol in cellular network for improving quality of service", MTAP 84:26267-26292 (2025). It proposes GEbPDCM: Golden Eagle-based Primal-Dual Congestion Management, for LTE networks using AODV routing over a mobile ad-hoc topology.

1.1 The problem it addresses

In a mobile ad-hoc network, some nodes end up relaying more traffic than others -- because they sit on more shortest paths, because of temporary clustering under mobility, or just by the random luck of the traffic pattern. Those "hot" nodes' buffers fill up, delay spikes, packets get dropped, and the node burns extra energy on retransmissions that still fail. Plain AODV has no mechanism to notice or correct this -- it only knows how to find a route, not how to relieve pressure once one exists.

1.2 The paper's proposed fix, restated as an algorithm

- For every node, continuously estimate two numbers: how much data it's currently carrying (its "load"), and how crowded its neighbourhood is (its "density").
- Combine load + density into a single congestion score.
- If a node's score crosses a threshold, it is "congested". Find another node nearby that is NOT congested, and move ("migrate") half of the congested node's load onto it.
- Repeat every time step. Over time, load smooths out across the network instead of piling up at a few hot nodes.

That's the entire actual algorithm. The paper's title promises two extra techniques -- a "Golden Eagle Optimizer" for picking WHICH neighbour to migrate to, and a "primal-dual" mathematical framing of the whole problem -- but neither appears anywhere in the paper's method section. This gap is not a minor omission; it's the central finding that shaped this whole project (see Section 10 and the Lab Report's Section 7.1). Building the GEO the paper claims but never wrote is this project's first required novelty; LTPM (trust + forecasting) is the second, original to this project.

1.3 The equations (paper numbering, as given to us)

- Eq 2: P = Nc + h(sum Dl) -- "predicted load". As literally written this adds a set of nodes (Nc) to a data-rate sum (Dl) -- not addable. Our reading: P = h x (sum of the node's recent load readings), i.e. a smoothed/monitored load estimate. h = 0.8, a scaling coefficient we chose.
- Eq 3: Nd = Dl + Nn(Pmax/Pt) -- "node density". Also mixes a load term with a neighbour-count term. Our reading: Nn = number of neighbours within range, (Pmax/Pt) = ratio of the network's max data rate to this node's current load (an "interference pressure" term) -- so Nd grows both with the node's own load and with how many neighbours it's contending against.
- Eq 4: Ns = (P + Nd) / (k x O_l) -- combine into one normalised "node status" score. O_l is the paper's fixed optimal load (325 Mbps in Table 1). k is a calibration constant (we use 1.6, chosen so Ns lands in a sensible range for this implementation's own traffic scale -- not a value from the paper).
- Eq 5: classify Ns (called CS in our code, congestion score) against two thresholds: below 0.5 = safe, 0.5 to 1.0 = at-threshold, at/above 1.0 = congested.
- Eq 6: U = Ns + P(CS) -- a "user priority" score for who gets to migrate first when multiple nodes are congested simultaneously.
- Eq 7: if U >= 1, this node is the "first user" -- top migration priority this tick.
- Eq 8: a node "shares" (migrates) once its predicted load P exceeds O_l.
- Eq 9-12: standard QoS metrics -- average delay, throughput, packet drop ratio, packet delivery ratio (PDR).

2. src/config.py

Every tunable number in the project lives here, in one place, split into two clearly labelled sections.

2.1 Section 1: values taken directly from the paper's Table 1

```
AREA_WIDTH = 1000; AREA_HEIGHT = 1000   # metres
NUM_NODES = 100
BASE_STATION = (500, 450)
TX_RANGE = 300              # metres, neighbour/transmission range
DATA_RATE_MIN = 3.1086      # Mbps
DATA_RATE_MAX = 49.0946     # Mbps
BANDWIDTH = 27.175          # Mbps
JITTER_MS = 6.4
SPEED_SWEEP = [10, 20, 30, 40, 50]
PACKET_SIZE_SWEEP = [1, 2, 3, 4, 5]
SIM_TIME = 100
```

Why keep these separate from our own tunables (Section 2 below)? Because under viva questioning you must be able to instantly say "this number is the paper's, I did not choose it" versus "this number is my own calibration, here's why I picked it." Mixing them would make that impossible to answer confidently.

2.2 Section 2: our own tunables

- CONGESTION_THRESHOLD = 0.5, CONGESTED_LEVEL = 1.0 -- the Eq5 boundaries, taken directly from the paper's own description of the three-way classification.
- OPTIMAL_LOAD = 325 (Mbps) -- the paper's O_l, used ONLY for Eq8's should_share() gate (does this node's predicted load exceed the paper's stated optimum), because that's the one place the paper gives O_l an explicit, unambiguous role.
- SEVERITY_REFERENCE_LOAD = 90 -- a SEPARATE reference used by the drop-rate/delay model (v2) and by GEO's fitness function (both versions). Why not just reuse O_l=325 everywhere? Because real per-node loads in this simulator's traffic model never get anywhere close to 325 Mbps -- comparing against O_l there would mean the 'severity' term never leaves the noise floor, and GEO would never be able to tell candidates apart. 90 was picked empirically so a typical node's load produces a believable spread of severity values. This is flagged explicitly in the code comments as OUR constant, not the paper's.
- MONITOR_COEFF_H = 0.8 -- Eq2's h coefficient, chosen so P stays roughly the same order of magnitude as OPTIMAL_LOAD.
- TRUST_INIT=0.5, TRUST_MIN=0.6, TRUST_ALPHA=0.9 -- LTPM's EWMA trust parameters. 0.9 is a standard EWMA smoothing factor (90% weight on history, 10% on the newest observation) -- slow enough that one bad tick doesn't instantly blacklist a node, fast enough that a persistently bad node falls below TRUST_MIN within a handful of ticks.
- UNRELIABLE_FRACTION = 0.10, INTRINSIC_FAIL_RATE = 0.3 -- 10% of nodes are seeded with an extra, congestion-independent chance of failing to forward. WHY: without this, every node's natural forwarding success rate is high enough that trust converges above TRUST_MIN for everyone, and LTPM's trust filter never actually excludes a single candidate in testing -- it would look implemented but be silently inert. This gives the mechanism something real to filter, so its effect is actually observable and testable.
- FORECAST_WINDOW = 5 -- LTPM's forecast uses the last 5 load readings. Short enough to react to a real trend, long enough that a single noisy tick doesn't dominate the linear fit.
- GEO_POPULATION = 20, GEO_ITERATIONS = 30 -- standard-sized swarm/iteration counts for a metaheuristic search at this problem scale (searching among at most a few dozen real neighbour candidates); large enough to explore, small enough to run every single congested tick without becoming the simulation's bottleneck.
- FITNESS_WEIGHTS: throughput 0.3, pdr 0.3, delay 0.2, loss 0.1, energy 0.1 (sums to 1.0) -- throughput and PDR are weighted highest because they are the metrics the paper cares most about; delay next; loss and energy lowest since they are more second-order effects of the other three.

2.3 Design-fork note

Why not fold everything into ONE reference load constant? Because O_l (325) and SEVERITY_REFERENCE_LOAD (90) answer two different questions -- "does this node exceed the paper's stated optimum" (a fixed target) vs. "how bad is this node's congestion relative to what THIS simulator's traffic actually produces" (a scale that has to match the implementation, not the paper's abstract number). Conflating them would make Eq8's gate almost never fire (loads rarely reach 325) while ALSO making the severity model's output meaningless. Keeping them separate, and documenting why, is deliberate.

3. src/topology.py

Defines the Node data structure and how the 100-node layout is obtained.

```
class Node:
    id, x, y, buffer, load, energy, trust, load_history, status, reliable
```

- load_topology(path): if data/topology.csv exists and is non-empty, load the real NS-3-exported positions from it. Otherwise, generate 100 positions uniformly at random over the paper's 1000x1000m area, using a FIXED seed (config.RANDOM_SEED = 42) so results are reproducible run-to-run.
- WHY the CSV path is the DEFAULT and random generation is the fallback, not the other way round: the project's whole point is to test against a genuine topology, not a synthetic one, whenever one is available. Falling back only when the file is missing means the pipeline never silently breaks if NS-3 hasn't been run yet, but always prefers the real data when it has.
- build_neighbours(nodes): for every node, list every other node within TX_RANGE (300m) using plain Euclidean distance. This exact same rule is reused inside ns3/gebpdcm.cc's NeighboursWithin() (Section 9) -- keeping the neighbour definition identical between v1 and v2 is what makes their results genuinely comparable despite everything else differing.
- distance(a, b): plain 2D Euclidean distance, ((a.x-b.x)**2 + (a.y-b.y)**2) ** 0.5.

4. src/status.py -- Eq 2-5

One function per equation, each docstring citing its equation number and stating our interpretation.

predict_data_load(node) -- Eq 2

```
def predict_data_load(node):
    window = node.load_history[-FORECAST_WINDOW:] or [node.load]
    return MONITOR_COEFF_H * sum(window)
```

Sums the node's last 5 load readings and scales by h=0.8. Using a WINDOW rather than just the single current reading is deliberate: Eq2 is called a PREDICTED load, and a sum over recent history is a natural, simple way to smooth out single-tick noise -- a lone spike doesn't instantly flip a node's status.

node_density(node, neighbours) -- Eq 3

```
def node_density(node, neighbours):
    nn = len(neighbours.get(node.id, []))
    pt = max(node.load, DATA_RATE_MIN)   # <-- floor, not 1e-6
    interference_ratio = DATA_RATE_MAX / pt
    return node.load + nn * interference_ratio
```

WHY floor Pt at DATA_RATE_MIN (3.1086) and not at some tiny epsilon like 1e-6? Because a near-idle node (load close to zero) would otherwise send DATA_RATE_MAX/Pt into the thousands, and ANY node with a handful of neighbours would read as permanently, absurdly congested regardless of actual traffic. Flooring at the paper's OWN stated minimum data rate keeps the interference ratio bounded at a sane maximum (about 49/3.1 = 15.8x) while still being a paper-derived number, not an arbitrary one we invented.

node_status(P, Nd) -- Eq 4

```
def node_status(P, Nd):
    return (P + Nd) / (1.6 * OPTIMAL_LOAD)
```

WHY 1.6 and not 2.0 (a more 'obvious' round number)? This was tuned empirically against this implementation's own traffic model: with a plain 2x factor, almost no node-ticks ever crossed into 'congested', making the whole migration mechanism a rare, hard-to-observe event. 1.6 was chosen so that 'congested' captures a believable MINORITY of node-ticks -- common enough to be measurable and demonstrable, not so common that the network is permanently saturated. This is explicitly flagged in the docstring as our calibration, not a paper value -- expect to be asked about it directly.

congestion_score(node, neighbours) -- Eq 5's score, and classify(cs) -- Eq 5's classification

```
def congestion_score(node, neighbours):
    P = predict_data_load(node)
    Nd = node_density(node, neighbours)
    return node_status(P, Nd)

def classify(cs):
    if cs < 0.5: return 'safe'
    if cs < 1.0: return 'threshold'
    return 'congested'
```

Straightforward composition of the three prior functions, then a simple threshold ladder exactly as the paper describes it (three bands, two cut points).

5. src/migrate.py -- Eq 6-8 + target dispatch

user_priority(Ns, P, cs) -- Eq 6

```
def user_priority(Ns, P, cs):
    cs_weight = min(max(cs - CONGESTED_LEVEL, 0) / CONGESTED_LEVEL, 1.0)
```

```
    return Ns + P * cs_weight
```

The paper's Eq6 is U = Ns + P(CS) -- 'P weighted by CS' without saying exactly how. Our reading: the weight should be ZERO for any node that isn't actually over the congested threshold (cs_weight clamped at 0 below CONGESTED_LEVEL), and rise to a maximum of 1.0 once cs has moved a full CONGESTED_LEVEL past the threshold. This makes U behave sensibly: nodes that are merely 'at threshold' don't get an inflated priority from a load term that shouldn't matter yet.

is_first_user(U) -- Eq 7, should_share(P) -- Eq 8, migrate_half(src, dst)

```
def is_first_user(U): return U >= 1
def should_share(P, optimal=OPTIMAL_LOAD): return P > optimal
def migrate_half(src, dst):
    moved = src.load * 0.5
    src.load -= moved
    dst.load += moved
    dst.buffer += moved
```

These three are implemented literally as the paper states them -- no reinterpretation needed, they're dimensionally consistent as written.

select_target(src, candidates, scheme, network) -- the dispatch

```
if scheme == 'gebpdcm':      return _nearest_free_node(src, candidates)
if scheme == 'gebpdcm_geo':  return geo.select(src, candidates, network)
if scheme == 'gebpdcm_ltpm':
    trusted = [c for c in candidates if ltpm.trusted(c)]
    return geo.select(src, trusted, network) if trusted else None
```

WHY import geo/ltpm INSIDE the function (lazy import) rather than at the top of the file? This was a deliberate build-order decision from early in the project: scheme 'gebpdcm' needed to run and be checkpointed BEFORE geo.py or ltpm.py existed yet. A top-level import would have crashed immediately even for the scheme that doesn't need them. Lazy imports let the file be usable incrementally as the project was built up module by module.

_nearest_free_node(src, candidates): filters candidates to those NOT status=='congested', then returns the one with minimum Euclidean distance to src. This is exactly the paper's own plain-threshold rule -- no optimizer, which is precisely the gap GEO (Section 6) is built to fill.

6. src/geo.py -- Golden Eagle Optimizer (our contribution)

The paper's title claims this but its method section never builds it. This module is a REAL implementation of the Golden Eagle Optimizer metaheuristic, applied to the specific problem of picking a migration target.

The biological metaphor, translated into code

- A hunting eagle simultaneously does two things: flies straight at its prey (ATTACK -- exploitation, converging on the best solution found so far), and circles around it (CRUISE -- exploration, searching nearby space in case a better angle exists).
- Early in a hunt, an eagle cruises more (explores); late in a hunt, it commits to the attack (exploits). GEO models this with two weights that shift linearly over the iteration budget: p_attack rises 0.5 -> 2.0, p_cruise falls 1.0 -> 0.5.

fitness(candidate_node, source_node)

```
headroom      = max(0, SEVERITY_REFERENCE_LOAD - candidate.load) / SEVERITY_REFERENCE_LOAD
pdr_proxy     = {'safe':1.0, 'threshold':0.5, 'congested':0.0}[candidate.status]
delay_proxy   = distance(source, candidate) / TX_RANGE
loss_proxy    = min(candidate.load / (2*SEVERITY_REFERENCE_LOAD), 1.0)
energy_proxy  = 1.0 - candidate.energy / 100.0
F = 0.3*headroom + 0.3*pdr_proxy - 0.2*delay_proxy - 0.1*loss_proxy - 0.1*energy_proxy
```

WHY proxies and not a full re-simulation of 'what would happen if we migrated here'? Running a full flow simulation for every one of 20 eagles x 30 iterations x every congested node x every tick would be computationally absurd. Instead, each term is a CHEAP, DEFENSIBLE stand-in built from the candidate's CURRENT real state: how much headroom it has before it hits the severity reference, its current safe/threshold/congested status, how far away it is (closer = cheaper to reach = lower delay), how loaded it already is, and how much energy it's already spent. This is explicitly documented in the code as a deliberate simplification, not hidden.

WHY headroom/loss_proxy are scored against SEVERITY_REFERENCE_LOAD and NOT the paper's O_l (325 Mbps)? Same reasoning as config.py Section 2: real per-node loads in this simulator never approach 325, so scoring against it would make headroom ~1.0 and loss_proxy ~0.0 for EVERY candidate -- GEO would be unable to distinguish any candidate from any other, defeating the entire point of an optimizer.

select(source, candidates, ...) -- the attack/cruise loop

```
eagles = [random starting positions, seeded at candidate node positions]
best = argmax(fitness) over candidates   # seed the initial best
for it in range(30):
    p_attack = 0.5 + 1.5 * it/29
    p_cruise = 1.0 - 0.5 * it/29
    for eagle in eagles:
        attack = unit_vector(best_pos - eagle_pos)
        cruise = perpendicular(attack), randomly mirrored left/right
        eagle_pos += (p_attack*attack + p_cruise*cruise) * step_size
        nearest = nearest real candidate node to eagle_pos
        if fitness(nearest) > fitness(best): best = nearest
return best
```

- WHY do eagles move through CONTINUOUS 2D space rather than jumping directly between candidate nodes? Because that's what makes it a genuine GEO search rather than just 'evaluate fitness of every candidate and pick the max' (which would need no optimizer at all, just a for-loop). The eagle explores the plane; at each step it's SCORED via the nearest real node to its current continuous position, since the thing we must ultimately return is an actual network node, not a point in space.
- WHY a random.Random(source.id) seeded RNG, not the shared global RNG? Determinism per source node: the same congested node, given the same candidates, produces the same search trajectory every time it's called within a run, which makes results reproducible and debuggable, without needing every node's search to consume from (and disturb) a single shared random stream.

- step_size = TX_RANGE / 10: an eagle can move at most 1/10th of the transmission range per iteration -- large enough to cross the whole search area within the 30-iteration budget, small enough not to overshoot wildly past the best position every step.

7. src/ltpm.py -- Lightweight Trust and Predictive Module (our contribution)

Two independent mechanisms addressing the two research gaps identified in the literature survey: security (Section 3.1) and proactive-vs-reactive (Section 3.1).

update_trust(node, success) and trusted(node) -- the trust half

```
def update_trust(node, success):
    a = TRUST_ALPHA   # 0.9
    node.trust = a*node.trust + (1-a) if success else a*node.trust
    node.trust = clamp(node.trust, 0, 1)

def trusted(node): return node.trust >= TRUST_MIN   # 0.6
```

This is a textbook EWMA (exponentially weighted moving average). WHY EWMA and not, say, a plain running average of all-time success rate? EWMA naturally forgets old history at a controlled rate (a=0.9 means each tick's weight decays by 10%), so a node that WAS reliable but has recently started failing gets penalised quickly, while a node that's currently reliable isn't held back forever by one bad tick early in the run. A plain average would react far too slowly to a node that degrades mid-simulation.

forecast_load(node, window) -- the predictive half

```
def forecast_load(node, window=5):
    history = node.load_history[-window:]
    if len(history) < 2: return history[-1] if history else node.load
    # ordinary least-squares fit of a straight line through the last `window` readings
    slope, intercept = linear_regression(x=[0..n-1], y=history)
    return slope * n + intercept   # one step past the last reading
```

should_preemptively_migrate(node)

```
current_severity  = max(0, node.load / SEVERITY_REFERENCE_LOAD - 0.5)
forecast_severity = max(0, forecast_load(node) / SEVERITY_REFERENCE_LOAD - 0.5)
return current_severity < 0.5 and forecast_severity >= 0.5
```

WHY linear extrapolation and not an LSTM / ARIMA / any trained model? Three reasons, and you should be ready to give all three under questioning: (1) the module is explicitly named LIGHTWEIGHT -- a trained model contradicts its own name; (2) a trained model needs training DATA and a training PROCESS neither the paper nor this project has any basis for -- it would be unjustifiable scope creep, not a real requirement; (3) a simple linear trend is fully sufficient for the actual job here, which is not accurate multi-step forecasting but a cheap early-warning signal one tick ahead. Reaching for a heavy model when arithmetic solves the problem is bad engineering, not good engineering.

WHY compare against the SAME 0.5 offset the aggregate drop-rate model (v2's simulator.py) actually reacts to, rather than some other threshold? So the 'preemptive' trigger fires exactly at the point real degradation is about to start being visible in the metrics -- any other threshold would be either too early (over-triggering, wasting migrations) or too late (no longer preemptive in any meaningful sense).

8. src/metrics.py and src/simulator.py (v2's engine)

metrics.py -- Eq 9-12 + energy model

```
def delay(total_delay, received): return total_delay / received if received else 0
def throughput(received, bits_per_pkt, time_s): return received*bits_per_pkt/time_s/1e6
def packet_drop_ratio(sent, received): return (sent-received)/sent if sent else 0
def pdr(sent, received): return received/sent if sent else 0

def energy(hops_tx, hops_rx):
    dropped = hops_tx - hops_rx
    return hops_tx*TX_COST + hops_rx*RX_COST + dropped*RETRY_COST
```

WHY charge a RETRY_COST for dropped hops specifically, on top of the tx/rx cost? Because a real radio still spends airtime and power attempting a transmission that ultimately fails -- a scheme that drops MORE should cost MORE energy, not less. Without this extra term, a scheme that merely drops everything immediately (spending almost no tx/rx effort) would look artificially cheap, which directly inverts the paper's own claim that congestion control SAVES energy. This term is what makes the energy graph tell a sensible story.

simulator.py -- the v2 tick loop, run(scheme, speed, packet_size_kb)

One tick = one simulated second, repeated SIM_TIME (100) times. Each tick, in order:

- 1. Move every node one second of random-waypoint mobility at the given speed.
- 2. Generate this tick's OFFERED traffic per node: a fresh random draw between DATA_RATE_MIN and DATA_RATE_MAX, added to that node's running load and load_history.
- 3. Compute every node's congestion score and classification (status.py).
- 4-5. If migration is enabled for this scheme, any node classified 'congested' (or, for gebpdcm_ltpm, flagged by should_preemptively_migrate) picks a target via migrate.select_target and migrate_half()'s.
- 6. Forward each node's current load one hop towards the base station; a CONTINUOUS severity formula (not just the discrete safe/threshold/congested bucket) drives that node's drop rate and queueing delay for this tick, and energy is deducted.
- 7. Accumulate running totals; after all 100 ticks, compute the Eq9-12 metrics from those totals and return them as one result dict.

WHY a SEPARATE trust_rng (random.Random(RANDOM_SEED+1)), not reusing the main rng, for LTPM's trust coin-flip? If LTPM's extra random draws came from the SAME stream as mobility/traffic generation, they would shift every SUBSEQUENT draw for that scheme only -- silently giving gebpdcm_ltpm a DIFFERENT sequence of node positions and offered traffic than the other three schemes see, breaking the fair same-topology-same-traffic comparison the whole experiment depends on. A separate stream keeps all four schemes' traffic/mobility identical, varying only in their congestion-control logic -- the actual thing being compared.

WHY divide offered traffic by PACKET_RATE_SCALE (30) when converting Mbps into a packet count for the tick? Treating a node's full offered Mbps as literally, fully delivered every single tick would put 100 nodes x up to 49 Mbps each into multi-gigabit aggregate throughput -- two orders of magnitude past anything the paper reports. Scaling down by this factor keeps the aggregate throughput in the same order of magnitude as the paper's own reported range (tens of Mbps), which is necessary for the comparison table (Section 6.6 of the Lab Report) to mean anything.

9. ns3/gebpdcm.cc -- the real NS-3 port (v1's engine)

Same equations, same GEO search, same LTPM logic as Sections 4-7 above -- ported into C++ and made to act on a REAL packet-level NS-3 simulation instead of an abstracted model. This is where most of the genuinely hard engineering decisions in the project were made, because NS-3 has no built-in concept that matches what the paper describes.

9.1 Scenario scaffold (topology / mobility / WiFi / AODV / traffic)

- 100 mobile nodes + 1 base station, RandomRectanglePositionAllocator over 1000x1000m, RandomWaypointMobilityModel at the sweep's speed -- matches config.py's parameters exactly so v1 and v2 run the identical scenario geometry.
- WHY is mobility set up BEFORE WiFi/Internet/AODV in main()? NS-3's position allocator draws from a SHARED global RNG stream. If WiFi/AODV setup ran first and consumed random draws of its own, the topology would end up depending on unrelated code elsewhere in main() -- fragile and non-reproducible. Running mobility first, before anything else touches the RNG, means the topology depends ONLY on RngSeedManager's seed/run, exactly as intended. (This was found the hard way, mid-debugging -- see Section 9.5.)
- WHY 802.11b with DataMode/ControlMode/NonUnicastMode = DsssRate2Mbps, not the originally-intended DsssRate11Mbps? A real bug in this specific ns-3.42 build (Section 9.5) silently discards every HR/DSSS (5.5/11 Mbps) frame as 'noise' at receive time. Plain DSSS (1/2 Mbps) is unaffected. This is a WORKAROUND for a genuine environment bug, not a design preference -- documented in the file's own header comment in detail.

9.2 The real load signal: SampleQueueLoadMbps()

```
Ptr<WifiNetDevice> wifiDev = DynamicCast<WifiNetDevice>(device);
Ptr<WifiMacQueue> queue = wifiDev->GetMac()->GetTxopQueue(AC_BE);
uint32_t nBytes = queue->GetNBytes();
return (nBytes * 8.0) / 1e6;   // bytes queued -> instantaneous Mbps
```

WHY the WiFi MAC transmit queue's byte count, and not (say) bytes sent by the node's OnOff application in the last tick? The OnOff app's rate is FIXED (12kbps/node) -- reading it back would just give a constant number, telling you nothing about actual congestion. The MAC queue's BACKLOG, by contrast, is the real physical signal of contention: it only builds up when the node can't get its packets onto the channel fast enough, which is exactly what 'congested' is supposed to mean. This is the direct, physically-grounded analogue of v2's synthetic node.load.

Design-fork note: the first attempt used NetDeviceQueueInterface::GetTxQueue(0)->GetNBytes() (the generic Byte-Queue-Limits-style interface many NetDevice types expose) -- this COMPILED but the class in this ns-3.42 build turned out not to actually have a GetNBytes() method on that particular queue type. WifiMac::GetTxopQueue(AC_BE), which returns the WiFi-specific WifiMacQueue (which DOES have GetNBytes()), was the fix. Small but real example of 'the generic API doesn't cover every subsystem -- check the specific one.'

9.3 The real migration mechanism: static-route override

NS-3 has no built-in "migrate this node's load to a neighbour" operation -- AODV computes its own routes and there is no per-node buffer to literally move bytes out of. The real-network EQUIVALENT of Eq6-8's migration was built as follows:

```
Ipv4StaticRoutingHelper srHelper;
Ptr<Ipv4StaticRouting> sr = srHelper.GetStaticRouting(srcNode->GetObject<Ipv4>());
sr->AddHostRouteTo(bsAddr, targetNeighbourAddr, wifiIfIndex);
uint32_t routeIdx = sr->GetNRoutes() - 1;
Simulator::Schedule(Seconds(0.5), &RemoveMigrationRoute, srcIdx, routeIdx);
```

This installs a per-node STATIC route that forces the congested node's traffic to the base station through the chosen neighbour instead of whatever route AODV had computed -- a REAL, observable change: actual packets now take a different physical path. It is removed again after 0.5 seconds (half the 1-second control tick), approximating Eq6-8's

"share 50% of load" as a bounded-duration reroute of real traffic rather than a literal byte split (which has no meaning once you're operating on real packets rather than a load NUMBER).

WHY does AODV alone need an explicit Ipv4StaticRoutingHelper added? By default, when you call internet.SetRoutingHelper(aodvHelper), that REPLACES the stack's routing entirely with just AODV -- there is no static-routing companion unless you build one yourself via Ipv4ListRoutingHelper. The very first attempt at this crashed with a segfault: GetStaticRouting() returned a null pointer (no static routing protocol existed on that node's stack at all), and calling ->GetNRoutes() on that null pointer segfaulted. The fix: explicitly combine Ipv4StaticRoutingHelper (priority 10) with AodvHelper (priority 0) via Ipv4ListRoutingHelper -- HIGHER priority wins in ns-3's list routing, so our override correctly takes precedence over AODV's own route while it's active, and AODV resumes control once it lapses.

9.4 The real trust signal: Tx/Drop packet traces

```
ipv4l3->TraceConnectWithoutContext("Tx", MakeBoundCallback(&TxTraceCallback, nodeIdx));
ipv4l3->TraceConnectWithoutContext("Drop", MakeBoundCallback(&DropTraceCallback, nodeIdx));
// each tick: success = (txThisTick > 0 && dropThisTick == 0); UpdateTrust(i, success);
```

WHY these two trace sources specifically, out of everything Ipv4L3Protocol exposes? "Tx" fires for every packet actually transmitted by that node's IP layer; "Drop" fires for every packet that layer discards (queue full, TTL expired, no route, etc). Together they give a REAL per-node, per-tick forwarding outcome -- exactly the signal LTPM's trust EWMA (Section 7) needs -- replacing v2's synthetic trust_rng.random() > drop_rate coin flip with an actual measured event.

9.5 The HR/DSSS bug -- found and fixed (the hardest debugging in the project)

- Symptom: 0% PDR in every configuration tried, including ns-3's own UNMODIFIED reference example (examples/routing/manet-routing-compare.cc) run for 200 seconds with its own defaults -- proof the bug was in the ns-3.42 build itself, not this project's scenario code.
- Method: rebuilt ns-3 in DEBUG profile (the default OPTIMIZED profile compiles NS_LOG statements out entirely -- no logging is possible in it) and re-ran with NS_LOG enabled on WifiPhy, YansWifiPhy and InterferenceHelper.
- Finding: the log showed 88,986 StartReceive events and ZERO EndReceive events, with the line "Unsupported modulation received (HR/DSSS), consider as noise" printed for every single one. The PHY REGISTERS HR/DSSS support during setup but then REJECTS every real HR/DSSS frame as noise at receive time -- a genuine bug in this specific ns-3.42 build.
- Fix: 802.11b's higher rates (5.5/11 Mbps) use HR/DSSS; only the LOWER rates (1/2 Mbps) use plain DSSS. Switching DataMode/ControlMode/NonUnicastMode from DsssRate11Mbps to DsssRate2Mbps moved all traffic onto the unaffected plain-DSSS path. A 3-second confirmation run went from 0% to 51.3% PDR.
- Consequence: the working channel capacity dropped from 11Mbps to 2Mbps, so the per-node OnOff traffic rate had to be retuned down proportionally (from 60kbps to 12kbps per node) to avoid oversubscribing the smaller channel 3x over. This is why v1's absolute delay/throughput numbers are lower than the paper's target range -- a consequence of the achievable channel, not a defect in the congestion-control logic.

10. Every Major Design Fork, in One Place

A quick-reference table for "why this, not that" -- everything below was an actual decision point faced during the project, not a hypothetical.

Decision point	Chosen	Rejected alternative	Why
Eq2/Eq3 dimensionally broken	Reinterpret & document our reading	Silently 'fix' without saying so	Hiding a reinterpretation as if it were the paper's own equation is dishonest and indefensible under questioning; documenting it as our finding is itself a valid contribution.
GEO target search	Real attack/cruise vector loop over continuous space, scored via nearest real node	Just rank candidates by a formula (no optimizer)	The paper's own title claims an optimizer; a ranking formula is exactly what the paper's un-optimized version already is.
LTPM forecasting model	Linear least-squares extrapolation	LSTM / ARIMA / trained ML model	Module is named 'Lightweight'; no training data or process exists to justify a trained model; the linear fit is fully sufficient for a 1-step-ahead early-warning signal.
Unreliable nodes	Seed 10% with an extra flat failure rate	Let reliability emerge naturally from congestion alone	Without an independent failure source, every node's trust converges above TRUST_MIN and the trust filter never excludes anyone -- it would look implemented but be inert.
v1's 'load' signal	Real WiFi MAC queue byte occupancy	Reuse v2's synthetic random draw inside NS-3 too	A synthetic draw ignores everything NS-3 actually simulates (real contention, real channel state) -- it would defeat the entire purpose of porting to a real simulator.
v1's migration mechanism	Bounded-duration Ipv4 static-route override	Literally move bytes between per-node buffers (impossible in a real network stack)	NS-3 has no per-node 'load' buffer to move bytes out of; a route override is the closest REAL, observable analogue of rerouting traffic.
v1's AODV + static routing	Explicit Ipv4ListRoutingHelper combining both, static at higher priority	AODV alone via SetRoutingHelper(aodv)	AODV alone has no static-routing companion at all -- GetStaticRouting() returns null and crashes; had to build the list explicitly.
v1's WiFi rate	DsssRate2Mbps (plain DSSS)	DsssRate11Mbps (HR/DSSS, originally intended, matches the paper's higher-throughput setting better)	This specific ns-3.42 build silently discards every HR/DSSS frame as noise at receive time -- a real, found-and-proven environment bug, not a preference.
v1's trust signal	Real Ipv4L3Protocol Tx/Drop packet traces	Keep the synthetic coin flip inside NS-3 too	Same reasoning as the load signal: a synthetic signal would waste the entire point of testing against a real simulator.
Reporting v1's 'baseline wins' result	Report it plainly, explain the mechanism (Lab Report 7.4)	Re-tune parameters until GEO/LTPM 'win' in NS-3 too	Tuning the result to match the paper's narrative would be scientifically dishonest and would hide the single most interesting finding of the whole project.

11. Anticipated Viva Questions

Q: Why does GEO/LTPM make things WORSE than baseline in the real NS-3 version, when your Python version shows them winning clearly?

A: Because v2 models capacity as independent per-node buckets (moving load off a node always helps, by construction), while v1's real network shares ONE physical radio channel among all 100 nodes -- rerouting through a neighbour adds an extra hop's worth of airtime on that SAME contended channel, it doesn't add capacity. This is a real finding about the paper's own design assumption, not a bug -- see Lab Report Section 7.4 for the full mechanism.

Q: If your NS-3 results show migration doesn't help, why keep GEO and LTPM in the project at all?

A: Because within the congestion-controlled schemes, GEO and LTPM still consistently beat plain nearest-free GEbPDCM at higher speeds in BOTH versions -- they are a genuine, measurable improvement over the paper's own design under identical constraints. The finding is that migration-as-a-strategy has a limit on a shared channel, not that smarter target selection is worthless.

Q: Why does the paper's title mention 'primal-dual' when there's no such formulation in your implementation?

A: Because there's none in the PAPER either -- we checked its method section in detail and found no Lagrangian, no dual variables, no primal-dual update rule anywhere. This is documented as a finding about the base paper (Lab Report Section 7.1), not an omission on our part.

Q: Why not just fix NS-3's 11Mbps rate issue and get results closer to the paper's numbers?

A: The bug (HR/DSSS frames discarded as noise) is a real defect in this specific ns-3.42 build's WiFi PHY, not a configuration choice -- switching to plain DSSS (2Mbps) is the actual fix that restores real packet delivery. Retuning offered traffic to fit that achievable capacity is honest engineering; artificially inflating the reported numbers to match the paper's target range would not be.

Q: Why did you build TWO versions instead of just picking one?

A: v2 (Python) let every equation's interpretation be worked out and validated quickly and cheaply, independent of a specific network simulator's quirks -- it's the fast, iterative design tool. v1 (NS-3) is the real test: does the algorithm actually work against a standards-compliant, packet-level, physically grounded network? Building both, and finding they DISAGREE, was the single most valuable outcome of the whole project -- a paper design flaw only visible once tested for real.

Q: What would you change if you had more time?

A: Test the congestion-control agent under a scenario with genuinely SEPARABLE capacity between migration targets -- e.g. multiple radio channels, or a wired backhaul per cluster -- which is the setting the paper's own migration model implicitly assumes, and under which GEO/LTPM's smarter target selection would be expected to close the gap to baseline in NS-3 too.